



RECETAS ESTRELLA

Presenta:

INTEGRANTES:

CHAVIRA RUIZ JAZMIN NC:24580007
DURAN TOLENTINO AMANDA NOHEMI NC:24580009
GARCIA ESPINOSA ANDREA NC:24580011
ROMERO RAMIREZ ALEXANDER NC:24580028
VILLALOBOS VILLARREAL NAIBY ADRIANA NC:24580914

Materia:

ingeniería de software

UNIDAD: 3,4,5

DOCUMENTACION DEL PROGRAMA PROYECTO DE BASE DE
DATOS Y PROGRAMACIÓN 2

Universidad:

TECNOLÓGICO NACIONAL DE MÉXICO

DOCUMENTACIÓN DE LA BASE DE DATOS:

Introducción

La base de datos del sistema Chef Recetas tiene como objetivo almacenar y administrar la información necesaria para el funcionamiento de la aplicación. Permite registrar usuarios, categorías, ingredientes y recetas, facilitando la consulta y administración de la información mediante tablas relacionadas y vistas de apoyo.

Objetivo

Diseñar e implementar una base de datos relacional que permita almacenar de forma organizada la información del sistema Chef Recetas, garantizando la integridad de los datos y facilitando las operaciones de consulta, registro, modificación y eliminación.

Herramientas Utilizadas

- MySQL Server
- MySQL Workbench
- Lenguaje SQL
- Visual Studio
- C#

Proceso de Creación de la Base de Datos

Paso 1. Creación de la Base de Datos

Se crea la base de datos denominada chef_recetas y posteriormente se selecciona para trabajar sobre ella.

```
CREATE DATABASE IF NOT EXISTS chef_recetas  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_spanish_ci;
```

```
USE chef_recetas;
```

Paso 2. Creación de la Tabla Usuarios

Esta tabla almacena la información de los usuarios que tendrán acceso al sistema.

Campos:

- id_usuario: Identificador único.
- nombre: Nombre completo.
- usuario: Nombre de acceso.
- password: Contraseña cifrada.

- rol: Tipo de usuario.
- activo: Estado del usuario.
- fecha_creacion: Fecha de registro.

Código SQL:

```
CREATE TABLE usuarios (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  usuario VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  rol VARCHAR(30) NOT NULL,
  activo BOOLEAN DEFAULT TRUE,
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Paso 3. Creación de la Tabla Categorías

Esta tabla almacena las categorías utilizadas para clasificar las recetas.

Campos:

- id_categoria
- nombre
- descripcion
- activo

Código SQL:

```
CREATE TABLE categorias (
  id_categoria INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL,
  descripcion TEXT,
  activo BOOLEAN DEFAULT TRUE
);
```

Paso 4. Creación de la Tabla Ingredientes

Esta tabla almacena el catálogo de ingredientes disponibles para las recetas.

Campos:

- id_ingredient
- nombre
- unidad_medida
- activo

Código SQL:

```
CREATE TABLE ingredientes (  
  id_ingredient INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  unidad_medida VARCHAR(30),  
  activo BOOLEAN DEFAULT TRUE  
);
```

Paso 5. Creación de la Tabla Recetas

Esta tabla contiene la información principal de cada receta registrada.

Campos:

- id_receta
- id_categoria
- nombre
- descripción
- tiempo_preparacion
- porciones
- dificultad
- fecha_creacion
- activo

Código SQL:

```
CREATE TABLE recetas (  
  id_receta INT AUTO_INCREMENT PRIMARY KEY,  
  id_categoria INT NOT NULL,  
  nombre VARCHAR(150) NOT NULL,  
  descripcion TEXT,  
  tiempo_preparacion INT,  
  porciones INT,  
  dificultad VARCHAR(20),  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  activo BOOLEAN DEFAULT TRUE,  
  CONSTRAINT fk_receta_categoria  
  FOREIGN KEY (id_categoria)  
  REFERENCES categorias(id_categoria)  
);
```

Creación de Vistas

Las vistas permiten realizar consultas más sencillas desde la aplicación desarrollada en C#.

Vista de Usuarios

```
CREATE OR REPLACE VIEW vw_usuarios AS
SELECT
    id_usuario,
    nombre,
    usuario,
    rol,
    activo,
    fecha_creacion
FROM usuarios;
```

Vista de Categorías

```
CREATE OR REPLACE VIEW vw_categorias AS
SELECT
    id_categoria,
    nombre,
    descripcion,
    activo
FROM categorias;
```

Vista de Ingredientes

```
CREATE OR REPLACE VIEW vw_ingredientes AS
SELECT
    id_ingredient,
    nombre,
    unidad_medida,
    activo
FROM ingredientes;
```

Vista de Recetas

```
CREATE OR REPLACE VIEW vw_recetas AS
SELECT
    r.id_receta,
    r.nombre,
    r.descripcion,
    r.tiempo_preparacion,
    r.porciones,
    r.dificultad,
    r.fecha_creacion,
    r.activo,
    c.nombre AS categoria
FROM recetas r
```

```
INNER JOIN categorias c
ON r.id_categoria = c.id_categoria;
```

Insertión del Usuario Administrador

Para poder acceder inicialmente al sistema se registra un usuario administrador.

Código SQL:

```
INSERT INTO usuarios
(nombre, usuario, password, rol, activo)
VALUES
(
    'Administrador',
    'admin',
    SHA2('123456',256),
    'Administrador',
    TRUE
);
```

Relaciones de la Base de Datos

Relación Categorías - Recetas

Una categoría puede contener muchas recetas, mientras que una receta solamente puede pertenecer a una categoría.

```
FOREIGN KEY (id_categoria)
REFERENCES categorias(id_categoria);
```

Consultas Utilizadas por el Sistema

Inicio de Sesión

La aplicación verifica que el usuario exista, que la contraseña coincida y que la cuenta se encuentre activa.

```
SELECT *
FROM usuarios
WHERE usuario=@u
AND password=SHA2(@p,256)
AND activo=1;
```

Listar Usuarios

```
SELECT *
FROM vw_usuarios
ORDER BY nombre;
```

Buscar Usuarios por Nombre

```
SELECT *  
FROM vw_usuarios  
WHERE nombre LIKE @c  
ORDER BY nombre;
```

Buscar Usuarios por Rol

```
SELECT *  
FROM vw_usuarios  
WHERE rol=@r  
ORDER BY nombre;
```

Contar Usuarios

```
SELECT COUNT(*)  
FROM usuarios;
```

Conclusión

La base de datos Chef Recetas fue desarrollada utilizando MySQL y está compuesta por las tablas Usuarios, Categorías, Ingredientes y Recetas. Además, utiliza vistas para facilitar las consultas realizadas desde la aplicación desarrollada en C#, permitiendo una administración eficiente y organizada de la información.

DOCUMENTACIÓN DEL CÓDIGO VISUAL STUDIO

1. Introducción

ChefRecetasCS es una aplicación desarrollada en C# con Windows Forms y MySQL para la administración de recetas de cocina. El sistema permite gestionar categorías, ingredientes, recetas, usuarios y reportes mediante una arquitectura por capas.

2. Objetivo

Desarrollar un sistema que facilite el registro, consulta, modificación y eliminación de recetas e ingredientes, permitiendo además la administración de usuarios y la generación de reportes.

3. Arquitectura del Sistema

El proyecto utiliza una arquitectura de tres capas:

Capa de Presentación (Views)

Contiene los formularios que interactúan con el usuario:

- FrmLogin
- FrmMain
- FrmCategorias
- FrmIngredientes
- FrmRecetas
- FrmUsuarios
- FrmReportes
- FrmDetalleReceta
- FrmRegistro

Estas ventanas permiten capturar información y mostrar resultados.

Capa de Controladores (Controllers)

Actúa como intermediaria entre la interfaz y la base de datos.

Controladores encontrados:

- CategoriaController
- IngredienteController
- RecetaController
- UsuarioController
- ReporteController

Capa de Acceso a Datos (DAL)

Se encarga de ejecutar consultas SQL en MySQL.

Clases:

- Conexion
- CategoriaDAL
- IngredienteDAL
- RecetaDAL
- RecetaIngredienteDAL
- PasoPreparacionDAL
- UsuarioDAL
- ReporteDAL

Capa de Modelos (Models)

Representa las entidades de la base de datos.

Modelos:

- Categoria
- Ingrediente
- Receta
- Usuario
- RecetaIngrediente
- PasoPreparacionReporte

4. Clase Conexion

La clase Conexion centraliza la conexión a la base de datos MySQL mediante una cadena de conexión que especifica servidor, puerto, base de datos, usuario y contraseña.

Método:

GetConexion()

Devuelve un objeto MySqlConnection listo para conectarse a la base de datos.

5. Controladores

CategoriaController

Funciones:

ObtenerTodas()

Guardar()

Eliminar()// Controlador encargado de gestionar las operaciones

```
// relacionadas con las categorías.

public class CategoriaController
{
    // Instancia de la capa de acceso a datos

    // para trabajar con la tabla Categorías.

    private readonly CategoriaDAL _dal = new CategoriaDAL();

    // Obtiene todas las categorías registradas

    // en la base de datos.

    public List<Categoria> ObtenerTodas()

        => _dal.Listar();
}
```

Su función es gestionar las categorías utilizando CategoriaDAL.

IngredienteController

Funciones:

- ObtenerIngredientes()
- GuardarIngrediente()
- EliminarIngrediente()

Permite administrar los ingredientes registrados.

RecetaController

Funciones:

- ObtenerTodas()
- ObtenerPorCategoria()
- Guardar()
- Eliminar()
- Buscar()

Permite administrar recetas y realizar búsquedas por nombre o descripción.

UsuarioController

Funciones:

- Login()
- Registrar()
- ObtenerUsuarios()
- ObtenerTotalUsuarios()
- GuardarUsuario()
- EliminarUsuario()

Administra el acceso y mantenimiento de usuarios.

ReporteController

Función:

- ObtenerReporte()

Obtiene información para los reportes de recetas.

```
/// <summary>
```

```
/// Controlador encargado de administrar los ingredientes.
```

```
/// Actúa como intermediario entre la interfaz y la capa DAL.
```

```
/// </summary>
```

```
public class IngredienteController
```

```
{
```

```
    /// <summary>
```

```
    /// Instancia de acceso a datos para ingredientes.
```

```
    /// </summary>
```

```
    private IngredienteDAL dao = new IngredienteDAL();
```

```
    /// <summary>
```

```
    /// Obtiene todos los ingredientes registrados.
```

```
    /// </summary>
```

```
    /// <returns>Lista de ingredientes.</returns>
```

```
public List<Ingrediente> ObtenerIngredientes()
{
    return dao.Listar();
}
```

```
/// <summary>
```

```
/// Guarda un ingrediente nuevo o actualiza uno existente.
```

```
/// </summary>
```

```
/// <param name="ingrediente">Ingrediente a guardar.</param>
```

```
public void GuardarIngrediente(Ingrediente ingrediente)
```

```
{
    // Si el ID es 0 se trata de un nuevo registro
    if (ingrediente.IdIngrediente == 0)
        dao.Insertar(ingrediente);
```

```
    // Si ya existe se actualiza
```

```
    else
        dao.Actualizar(ingrediente);
```

```
}
```

```
/// <summary>
```

```
/// Elimina un ingrediente por su identificador.
```

```
/// </summary>
```

```
/// <param name="id">ID del ingrediente.</param>
```

```
public void EliminarIngrediente(int id)
```

```
{
```

```
        dao.Eliminar(id);  
    }  
}
```

6. Módulo de Categorías

La clase CategoriaDAL implementa operaciones CRUD para las categorías.

```
/// <summary>  
/// Representa una categoría de recetas.  
/// </summary>  
  
public class Categoria  
{  
    /// <summary>  
    /// Identificador único de la categoría.  
    /// </summary>  
  
    public int IdCategoria { get; set; }  
  
    /// <summary>  
    /// Nombre de la categoría.  
    /// </summary>  
  
    public string Nombre { get; set; }  
  
    /// <summary>  
    /// Descripción de la categoría.  
    /// </summary>  
  
    public string Descripcion { get; set; }
```

```

    /// <summary>

    /// Nombre del icono asociado.

    /// </summary>

    public string Icono { get; set; }

}

```

Métodos:

- Listar()
- ObtenerPorFecha()
- ObtenerPorNombre()
- ObtenerEstadisticas()
- ObtenerDetalle()
- Insertar()
- Actualizar()
- Eliminar()/// <summary>

```

/// Clase encargada de realizar todas las operaciones

```

```

/// relacionadas con la tabla Categorías.

```

```

/// </summary>

```

```

public class CategoriaDAL

```

```

{

```

```

    /// <summary>

```

```

    /// Obtiene todas las categorías registradas.

```

```

    /// </summary>

```

```

    /// <returns>Lista de categorías.</returns>

```

```

    public List<Categoria> Listar()

```

```

    {

```

```

        List<Categoria> lista = new List<Categoria>();

```

```

        // Abre la conexión con MySQL

```

```

        using (var cn = Conexion.GetConexion())

```

```

{
    cn.Open();

    // Consulta SQL para obtener todas las categorías
    string sql = "SELECT * FROM categorias";
    using (var cmd = new MySqlCommand(sql, cn))
    using (var dr = cmd.ExecuteReader())
    {
        while (dr.Read())
        {
            lista.Add(new Categoria
            {
                IdCategoria = Convert.ToInt32(dr["id_categoria"]),
                Nombre = dr["nombre"].ToString(),
                Descripcion = dr["descripcion"].ToString()
            });
        }
    }

    return lista;
}

/// <summary>
/// Inserta una nueva categoría.
/// </summary>

/// <param name="c">Categoría a registrar.</param>
public void Insertar(Categoria c)
{

```

```

using (var cn = Conexion.GetConexion())
{
    cn.Open();

    string sql =

    @"INSERT INTO categorias

    (nombre, descripcion)

    VALUES

    (@nombre, @descripcion)";

    using (var cmd = new MySqlCommand(sql, cn))
    {
        // Parámetro para evitar inyección SQL

        cmd.Parameters.AddWithValue("@nombre", c.Nombre);

        cmd.Parameters.AddWithValue("@descripcion", c.Descripcion);


        cmd.ExecuteNonQuery();

    }
}

/// <summary>
/// Actualiza una categoría existente.
/// </summary>

public void Actualizar(Categoria c)
{
    using (var cn = Conexion.GetConexion())
    {
        cn.Open();
    }
}

```



```

        string sql =
            @"UPDATE categorias
            SET nombre=@nombre,
                descripcion=@descripcion
            WHERE id_categoria=@id";
        using (var cmd = new MySqlCommand(sql, cn))
        {
            cmd.Parameters.AddWithValue("@id", c.IdCategoria);
            cmd.Parameters.AddWithValue("@nombre", c.Nombre);
            cmd.Parameters.AddWithValue("@descripcion", c.Descripcion);
            cmd.ExecuteNonQuery();
        }
    }
}

/// <summary>
/// Elimina una categoría según su ID.
/// </summary>
public void Eliminar(int id)
{
    using (var cn = Conexion.GetConexion())
    {
        cn.Open();

        string sql =
            "DELETE FROM categorias WHERE id_categoria=@id";
    }
}

```

```

        using (var cmd = new MySqlCommand(sql, cn))
        {
            cmd.Parameters.AddWithValue("@id", id);

            cmd.ExecuteNonQuery();
        }
    }
}
}

```

7. Módulo de Ingredientes

La clase IngredienteDAL permite administrar ingredientes y sus imágenes.

Métodos:

- Listar()
- ObtenerPorFecha()
- ObtenerPorNombre()
- ObtenerEstadisticas()
- ObtenerDetalle()
- Insertar()
- Actualizar()
- Eliminar()

```
/// <summary>
```

```
/// Representa un ingrediente utilizado en las recetas.
```

```
/// </summary>
```

```
public class Ingrediente
```

```
{
```

```
    /// <summary>
```

```
    /// Identificador único del ingrediente.
```

```
    /// </summary>
```

```
    public int IdIngrediente { get; set; }
```

```

    /// <summary>

    /// Nombre del ingrediente.

    /// </summary>

    public string Nombre { get; set; }


    /// <summary>

    /// Unidad de medida.

    /// Ejemplo: gramos, litros, cucharadas.

    /// </summary>

    public string UnidadMedida { get; set; }


    /// <summary>

    /// Calorías por unidad.

    /// </summary>

    public decimal CaloriasPorUnidad { get; set; }


    /// <summary>

    /// Ruta de la imagen del ingrediente.

    /// </summary>

    public string ImagenPath { get; set; }
}

```

8. Módulo de Recetas

La clase RecetaDAL administra toda la información de recetas.

Métodos:

- Listar()
- ListarPorCategoria()
- Insertar()
- Actualizar()
- Eliminar()
- Mapear()

```
/// <summary>
```

```
/// Controlador encargado de administrar las recetas.
```

```
/// </summary>
```

```
public class RecetaController
```

```
{
```

```
    /// <summary>
```

```
    /// Instancia de acceso a datos para recetas.
```

```
    /// </summary>
```

```
    private readonly RecetaDAL _dal = new RecetaDAL();
```

```
    /// <summary>
```

```
    /// Obtiene todas las recetas registradas.
```

```
    /// </summary>
```

```
    /// <returns>Lista de recetas.</returns>
```

```
    public List<Receta> ObtenerTodas()
```

```
        => _dal.Listar();
```

```
    /// <summary>
```

```
    /// Obtiene recetas filtradas por categoría.
```

```
    /// </summary>
```

```
    /// <param name="idCat">ID de la categoría.</param>
```

```
    /// <returns>Lista de recetas.</returns>
```

```
    public List<Receta> ObtenerPorCategoria(int idCat)
```

```
        => _dal.ListarPorCategoria(idCat);
```

```
    /// <summary>
```

```
/// Guarda una receta nueva o actualiza una existente.
```

```
/// </summary>
```

```
/// <param name="r">Objeto receta.</param>
```

```
public void Guardar(Receta r)
```

```
{
```

```
    // Si no tiene ID se inserta
```

```
    if (r.IdReceta == 0)
```

```
        _dal.Insertar(r);
```

```
    // Si ya existe se actualiza
```

```
    else
```

```
        _dal.Actualizar(r);
```

```
}
```

```
/// <summary>
```

```
/// Elimina una receta de la base de datos.
```

```
/// </summary>
```

```
/// <param name="id">ID de la receta.</param>
```

```
public void Eliminar(int id)
```

```
    => _dal.Eliminar(id);
```

```
/// <summary>
```

```
/// Busca recetas por nombre o descripción.
```

```
/// </summary>
```

```
/// <param name="texto">Texto de búsqueda.</param>
```

```
/// <returns>Lista de coincidencias.</returns>
```

```
public List<Receta> Buscar(string texto)
```

```
{
```

```
        texto = texto.ToLower();

        return _dal.Listar().FindAll(r =>

            r.Nombre.ToLower().Contains(texto) ||

            r.Descripcion.ToLower().Contains(texto));

    }

}
```

9. Relación Recetas e Ingredientes

La clase RecetaIngredienteDAL administra la relación entre recetas e ingredientes.

Métodos:

- ObtenerTodos()
 - ObtenerPorReceta()
 - ObtenerEstadisticas()
 - ObtenerDetalle()
 - Mapear()
-

10. Pasos de Preparación

La clase PasoPreparacionDAL permite consultar instrucciones de elaboración de recetas.

Métodos:

- ObtenerTodos()
- ObtenerPorReceta()
- ObtenerEstadisticas()
- ObtenerDetalle()
- Mapear()

11. Gestión de Usuarios

La clase UsuarioDAL administra la seguridad y autenticación del sistema. Las contraseñas se almacenan utilizando SHA-256.

Métodos principales:

Autenticar()

Registrar()

ObtenerUsuarios()

ObtenerPorFecha()

ObtenerPorNombre()

ObtenerPorRol()

ObtenerEstadisticas()

ObtenerDetalle()

ObtenerTotal()

ActualizarUsuario()

EliminarUsuario()

GuardarUsuario()

/// <summary>

/// Controlador encargado de la administración de usuarios

/// y del proceso de autenticación.

/// </summary>

public class UsuarioController

{

/// <summary>

/// Instancia de acceso a datos para usuarios.

/// </summary>

private UsuarioDAL dao = new UsuarioDAL();

/// <summary>

/// Verifica las credenciales de un usuario.

/// </summary>

/// <param name="usuario">Nombre de usuario.</param>

```
/// <param name="password">Contraseña.</param>
```

```
/// <returns>Objeto Usuario autenticado.</returns>
```

```
public Usuario Login(string usuario, string password)
```

```
{
```

```
    return dao.Autenticar(usuario, password);
```

```
}
```

```
/// <summary>
```

```
/// Registra un nuevo usuario.
```

```
/// </summary>
```

```
public void Registrar(
```

```
    string nombre,
```

```
    string usuario,
```

```
    string password,
```

```
    string rol,
```

```
    bool activo)
```

```
{
```

```
    dao.Registrar(
```

```
        nombre,
```

```
        usuario,
```

```
        password,
```

```
        rol,
```

```
        activo);
```

```
}
```

```
/// <summary>
```

```
/// Obtiene todos los usuarios registrados.
```

```
/// </summary>
```



```
public List<Usuario> ObtenerUsuarios()
```

```
{
```

```
    return dao.ObtenerUsuarios();
```

```
}
```

```
/// <summary>
```

```
/// Obtiene la cantidad total de usuarios.
```

```
/// </summary>
```

```
public int ObtenerTotalUsuarios()
```

```
{
```

```
    return dao.ObtenerTotal();
```

```
}
```

```
/// <summary>
```

```
/// Guarda o actualiza un usuario.
```

```
/// </summary>
```

```
public void GuardarUsuario(
```

```
    int id,
```

```
    string nombre,
```

```
    string usuario,
```

```
    string password,
```

```
    string rol,
```

```
    bool activo)
```

```
{
```

```
    dao.GuardarUsuario(
```

```
        id,
```

```
        nombre,
```

```
        usuario,
```

```

        password,

        rol,

        activo);
    }

    /// <summary>
    /// Elimina un usuario.
    /// </summary>

    /// <param name="id">ID del usuario.</param>

    public void EliminarUsuario(int id)
    {
        dao.EliminarUsuario(id);
    }
}

```

12. Reportes

La clase ReporteDAL genera reportes sobre recetas y estadísticas del sistema.

Métodos:

- ObtenerReporteRecetas()
- ObtenerRecetasPorFecha()
- ObtenerRecetasPorNombre()
- ObtenerEstadisticaRecetas()
- ObtenerDetalleReceta()

```

    /// <summary>
    /// Clase encargada de generar reportes y estadísticas
    /// relacionadas con las recetas del sistema.
    /// </summary>

    public class ReporteDAL
    {

```

```

/// <summary>

/// Obtiene un listado completo de recetas para

/// mostrar en los reportes.

/// </summary>

/// <returns>Lista de recetas.</returns>

public List<Receta> ObtenerReporteRecetas()
{
    // Lista donde se almacenarán los resultados

    List<Receta> lista = new List<Receta>();

    // Apertura de la conexión a MySQL

    using (var cn = Conexion.GetConexion())
    {
        cn.Open();

        // Consulta SQL para obtener la información

        // que será mostrada en el reporte

        string sql =

        @"SELECT *

        FROM recetas

        ORDER BY nombre";

        using (var cmd = new MySqlCommand(sql, cn))

        using (var dr = cmd.ExecuteReader())
        {
            // Recorre cada registro obtenido

            while (dr.Read())
            {

                lista.Add(new Receta

```

```

    {
        // Asigna los datos obtenidos

        // a las propiedades del objeto

        IdReceta = Convert.ToInt32(dr["id_receta"]),

        Nombre = dr["nombre"].ToString(),

        Descripcion = dr["descripcion"].ToString(),

        TiempoPreparacion = Convert.ToInt32(dr["tiempo_preparacion"]),

        Porciones = Convert.ToInt32(dr["porciones"])

    });
}

}

}

// Regresa la lista para generar el reporte

return lista;

}

}

```

Conclusión

El sistema ChefRecetasCS implementa correctamente una arquitectura por capas que separa la presentación, la lógica de negocio y el acceso a datos. Esta estructura facilita el mantenimiento del código, la reutilización de componentes y la escalabilidad futura de la aplicación. Además, incorpora mecanismos de autenticación, generación de reportes y operaciones CRUD completas para la administración de recetas, ingredientes, categorías y usuarios.